Memory Compaction & Garbage Collector Engine

A C++ Runtime Memory Management Engine & LISP2 Compaction System

Author / System: Autonomous C++ Garbage Collector Engine & Runtime Simulator
Date: October 2026 | **Version:** 1.2 Production Release | **Platform:** macOS / POSIX

1. Executive Summary & Problem Formulation

Modern managed runtime environments—such as the **Java Virtual Machine (JVM)**, **V8 (Google Chrome/Node.js)**, and the **.NET CLR**—rely on autonomous dynamic memory management to prevent memory leaks and dangling pointers. However, simple Mark-and-Sweep allocators suffer from a severe architectural limitation: **External Memory Fragmentation**.

Over time, objects of varying sizes are allocated and deallocated at scattered locations. Although total free memory across the heap may far exceed the requirements of an incoming allocation, the free memory is fragmented into disjoint, sub-allocation-size gaps. As a result, an allocation request fails—triggering an unexpected *OutOfMemoryError*.

This project implements an end-to-end, high-performance in-memory runtime engine written in standard C++17. It couples continuous 1D heap memory tracking with a **Live Object Binary Search Tree**, a stack-based DFS Mark-and-Sweep collector, and a full **LISP2 Three-Pass Memory Compactor** that relocates surviving objects, adjusts interior reference pointers, and coalesces fragmented free holes into one continuous block.

Key Highlight & Unique Architectural Feature:
This engine explicitly demonstrates how *Two-Finger / LISP2 Memory Compaction* resolves fatal allocation deadlocks. When total free heap memory is **212 Bytes**, an incoming allocation of **100 Bytes** fails because the largest contiguous hole is only **40 Bytes**. After running compaction, all live blocks slide to low memory, consolidating a single **212 Byte** contiguous gap, allowing the 100 Byte request to succeed without increasing total heap capacity.

2. Core Architecture & Data Structures

The engine operates entirely in user-space memory without third-party dependencies, leveraging pure C++ standard library primitives and POSIX sockets:

Component	Data Structure	Role & Algorithmic Complexity
Heap Array	`uint8_t heap[512]`	Contiguous 1D byte array. Value 0 signifies unallocated space; non-zero values correspond to the owning Object ID.
Live Object Index	`ObjectBST (Self-contained BST)`	Maintains live objects sorted by starting memory byte offset. Guarantees *O(log N)* point lookup, in-order address sorting, and rapid gap identification.

Object Registry	`std::map<int, HeapObject>`	Stores metadata: unique ID, type tag (e.g., *Array, Buffer*), byte offset, size, root pointer flag, and directed reference adjacency list.
Reachability Graph	`std::stack<int> DFS Traversal`	Traces live reachable references beginning at Root references (simulating thread stacks/registers) to identify non-garbage nodes.
LISP2 Compactor	`Three-Pass Two-Finger Pointer Relocator`	Calculates destination offsets, patches graph references, and shifts data blocks to eliminate fragmentation completely.
POSIX HTTP Server	`Non-blocking BSD Sockets`	Lightweight embedded REST API server handling CORS preflight, serving UI assets, and JSON RPC requests.

3. Detailed Algorithmic Execution Pipeline

The engine operates a multi-stage memory management cycle. Each phase can run either independently or as part of a full automated cycle:

Phase 1: Mark Phase (Stack-Based DFS Reachability)

At GC invocation, every live object in objects has its marked flag reset to false. The engine identifies all **Root Objects** (objects with is_root == true, corresponding to active local variables, globals, or registers). A depth-first traversal is executed using an explicit std::stack<int>. For every popped node, all outgoing edges in its references vector are inspected; unvisited targets are pushed onto the stack and marked reachable. Any object unreachable from the root set remains unmarked.

Phase 2: Sweep Phase (Object Reclamation & Hole Generation)

The engine scans the object registry. For every node with marked == false:
1. Its physical byte range [offset ... offset + size - 1] in the heap array is wiped to 0.
2. The object node is deleted from the ObjectBST.
3. Any references pointing to this dead node from surviving objects are purged.
4. The object descriptor is erased from memory.
Notice: Sweeping clears dead bytes in place, but does **not** shift surviving objects. This creates an alternating sequence of occupied blocks and free holes.

Phase 3: LISP2 Compaction Algorithm (Offset & Pointer Adjustment)

To resolve fragmentation without external memory buffers, the classic **LISP2 Compactor** runs in three disciplined sequential passes:

- **Pass 1 — Compute Forwarding Addresses:** Traverses surviving objects in ascending memory offset order (via bst.inorder()). Maintains an allocation watermark free_ptr starting at 0. Assigns obj.forward_addr = free_ptr and advances free_ptr += obj.size.
- **Pass 2 — Update Inter-Object References (Pointer Adjustment):** In a real JVM or V8 runtime, physical pointer references held by surviving objects are updated to point to the new destination addresses computed in Pass 1. In our engine, this pass ensures that structural relationships remain coherent across object movement.
- **Pass 3 — Relocate Live Data Blocks:** Copies each surviving object's memory block from its original offset to its forward_addr. Wipes the remaining high-memory space [free_ptr ... HEAP_SIZE - 1] to 0. Rebuilds the BST with updated offsets.

4. Step-by-Step Mathematical Proof of Compaction

Below is the exact byte-level breakdown of the checkerboard fragmentation scenario built into /api/demo and verified by the test suite:

Obj ID	Type	Size	Offset Range	Status	Post-Sweep State	Post-Compact Offset
1	Array	40 B	`0 – 39`	**Live (Root)**	Retained	`0 – 39 (0x000)`
2	Buffer	32 B	`40 – 71`	Dead (Garbage)	**Reclaimed (Gap 1)**	Merged
3	HashMap	48 B	`72 – 119`	**Live (Root)**	Retained	`40 – 87 (0x028)`
4	String	36 B	`120 – 155`	Dead (Garbage)	**Reclaimed (Gap 2)**	Merged
5	Vector	32 B	`156 – 187`	**Live (Root)**	Retained	`88 – 119 (0x058)`
6	Closure	40 B	`188 – 227`	Dead (Garbage)	**Reclaimed (Gap 3)**	Merged
7	Promise	44 B	`228 – 271`	**Live (Root)**	Retained	`120 – 163 (0x078)`
8	RegExp	36 B	`272 – 307`	Dead (Garbage)	**Reclaimed (Gap 4)**	Merged
9	WeakMap	40 B	`308 – 347`	**Live (Root)**	Retained	`164 – 203 (0x0a4)`
10	Symbol	32 B	`348 – 379`	Dead (Garbage)	**Reclaimed (Gap 5)**	Merged
11	Iterator	48 B	`380 – 427`	**Live (Root)**	Retained	`204 – 251 (0x0cc)`
12	Proxy	36 B	`428 – 463`	Dead (Garbage)	**Reclaimed (Gap 6)**	Merged
13	Anchor	48 B	`464 – 511`	**Live (Root)**	Retained	`252 – 299 (0x0fc)`

Metric	State 1: Initial Allocation	State 2: After Sweep (Fragmented)	State 3: After LISP2 Compaction
Used Memory	512 Bytes (100%)	300 Bytes (58.6%)	300 Bytes (58.6%)
Total Free Memory	0 Bytes (0%)	**212 Bytes** (41.4%)	**212 Bytes** (41.4%)
Largest Free Block	0 Bytes	**40 Bytes**	**212 Bytes** (Single block)
Fragmentation Index	0.0%	**81.1%**	**0.0%** (Zero fragmentation)
Allocate 100B Result	FAIL (No memory)	**FAIL (Fragmented)**	**SUCCESS (Allocated ID 14)**

5. Embedded HTTP REST API Specification

The engine integrates an asynchronous HTTP server listening by default on port 8080. It supports standard CORS (Access-Control-Allow-Origin: *), JSON request payloads, and preflight OPTIONS requests:

Method & Route	Request Body	Response Payload	Description

`GET /` `GET /index.html`	*None*	HTML Web Visualizer	Renders live UI with 64x8 heap grid, BST tree canvas, graph canvas, and event log.
`GET /api/state`	*None*	Full JSON engine state	Returns 512-byte array, object metadata, reference lists, BST nodes, stats, and last 100 log lines.
`POST /api/allocate`	`{"name":"Buffer", "size":64, "is_root":true, "with_gc":false}`	{"success":true, "id":14} or failure with gap stats	Allocates byte range. If with_gc is true, automatically triggers GC cycle upon failure.
`POST /api/reference`	`{"from":1, "to":5}`	{"success":true}	Adds a directed reference link between two active objects for reachability analysis.
`POST /api/remove-root`	`{"id":2}`	{"success":true}	Clears the root bit, converting node into collectible garbage if unreferenced.
`POST /api/gc`	`{"phase":"full"} | "mark" | "sweep" | "compact"`	Updated state JSON	Triggers complete Mark-Sweep-Compact or executes an individual phase for inspection.
`POST /api/demo`	*None*	Updated state JSON	Sets up the verified 13-object checkerboard fragmentation dilemma.
`POST /api/reset`	*None*	Updated state JSON	Flushes all objects, wipes heap array to zero, and resets internal counters.

6. Verification & Automated Test Commands

To verify this runtime engine on any Unix/POSIX system, execute the provided automated walkthrough script or cURL tests:

```
# 1. Launch backend daemon on port 8080
./gc_engine 8080

# 2. Run the automated colored terminal demonstration
python3 demo_walkthrough.py

# 3. Open the rich visualization dashboard in any browser
open http://localhost:8080
```

Summary Conclusion: The Memory Compaction & Garbage Collector Engine provides a comprehensive, rigorous simulation of production-grade JVM and V8 heap management. By marrying an explicit Binary Search Tree live-object index with the three-pass LISP2 Compaction algorithm, it completely resolves fragmentation bottlenecks and ensures maximum memory utilization.